HUMAN TECHNOPOLE

2. Modules and subworkflows

Nextflow from zero to hero

2026/01/24

Modularity
and why it is your friend

- **Problem**: monolitic pipelines are hard to scale and maintain

- **Solution**: modularity
  - Maintainability
  - Code reuse across pipelines
  - Independent testing of components
  - Clear code organization
  - Community collaboration

Variant calling pipeline

FASTP

FASTQC

BWA_MEM

SAMTOOLS_SORT

SAMTOOLS_INDEX

SAMTOOLS_STATS

MOSDEPTH

DEEPVARIANT

MULTIQC

Scope of the “box”

Modules and Subworkflows

- **Module**

Process that is stored in a **separate file** and can be imported into different workflows.

- **Subworkflow**

Higher-level building block that groups **multiple modules** into a **logical unit of execution**, also in a separate file.

Variant calling pipeline

FASTP	READS_QC
FASTQC	
BWA_MEM	MAPPING
SAMTOOLS_SORT	
SAMTOOLS_INDEX	
SAMTOOLS_STATS	MAPPING_STATS
MOSDEPTH	
DEEPVARIANT	VARIANT_CALL
MULTIQC	REPORTING

Creating a module

- Put the process in a separate, independent file with extesion *.nf* (ie. *bwa.nf*)

- Input, logic and output of the module are controlled with `input`, `script` and `output` in the process definition.

```
process BWA_MEM {

    input:
    tuple val(sample_id), val(fastq_set_id), path(fastq_R1), path(fastq_R2), path(genome_fasta)

    output:
    tuple val(sample_id), path("${sample_id}-${fastq_set_id}.bwa.bam"), emit: bam_file
    tuple val(sample_id), path("${sample_id}-${fastq_set_id}.bwa.log"), emit: bwa_log

    script:
    """
    bwa mem -t 4 \
        -R \"@RG\\tID:${sample_id}\\tSM:${sample_id}\\tPL:Illumina\" \
        ${genome_fasta} \
        ${fastq_R1} ${fastq_R2} \
        2> ${sample_id}-${fastq_set_id}.bwa.log \
        | samtools view --threads 4 -Sb - > ${sample_id}-${fastq_set_id}.bwa.bam
    """
}
```

Creating a subworkflow

- **Import** the modules using `include`.
- Create a **named workflow**.
- Input, logic and output of the subworkflow are controlled with `take`, `main` and `emit` in the workflow definition.

```
include { BWA_MEM } from './bwa'
include { SAMTOOLS_SORT } from './samtools-sort'
include { MOSDEPTH } from './mosdepth'


Preview DAG
workflow MAPPING {

    take:
        qc_reads
        genome_indexes

    main:
        bwa_input_ch = qc_reads.combine(genome_indexes)
        BWA_MEM(bwa_input_ch)
        bam_files_ch = BWA_MEM.out.bam_file

        SAMTOOLS_SORT(bam_files_ch)

    emit:
        sorted_bam = SAMTOOLS_SORT.out.sorted_bam
}
```